

Experiment 4

Problem Statement:

Perform Exploratory Data Analysis (EDA) using Matplotlib to visualize trends, patterns, and anomalies in a given dataset. Generate bar charts and line graphs to uncover insights. Present findings through a visualization dashboard or report, summarizing key observations.

Theory:

Exploratory Data Analysis (EDA) is a critical step in data preprocessing, where analysts explore datasets to identify patterns, trends, and anomalies. EDA helps in understanding data distribution, relationships between variables, and potential issues like missing values or outliers..

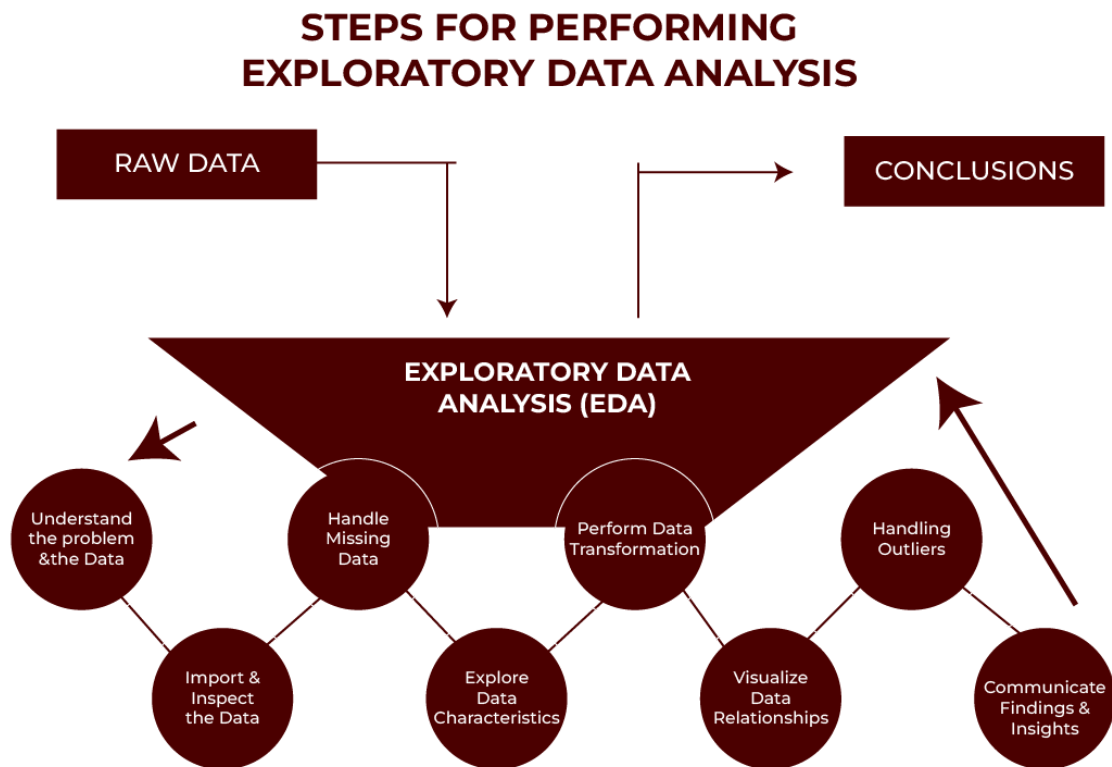
Objectives of EDA

- Understanding Data Structure – Identifying data types, missing values, and summary statistics.
- Detecting Trends & Patterns – Observing general trends over time or categories.
- Identifying Anomalies – Spotting unusual data points (outliers) that may affect analysis.
- Visualizing Relationships – Using plots to examine correlations and distributions.

Steps of EDA:

- 1. Understand the Problem & the Data** - Before starting any data analysis, it is crucial to understand the problem statement and the dataset. This involves defining the objectives of the analysis, understanding the business or domain-specific requirements, and identifying the key variables that influence the outcome. Knowing the nature of the data, such as whether it is structured (tables, databases) or unstructured (text, images), helps in selecting the right techniques for analysis. A good understanding of the data ensures that meaningful insights are extracted, avoiding misinterpretations.
- 2. Import & Inspect the Data** - Once the problem is understood, the next step is to import the dataset into a suitable tool such as Python (using Pandas) or R. Inspecting the dataset includes checking its structure, data types, and general statistics. Functions like `.head()`, `.info()`, and `.describe()` help in understanding the number of records, missing values, and distribution of variables. This step also involves checking for duplicate entries, inconsistent formats, and basic data integrity, ensuring that the dataset is ready for deeper analysis.

3. **Handle Missing Data** - Data is often incomplete due to various reasons, such as human errors or system failures. Handling missing values is a critical step in EDA to ensure accurate analysis. Missing data can be treated by either removing the affected rows/columns (`dropna()`) or filling them with appropriate values (`fillna()`). The imputation strategy depends on the type of data—mean, median, or mode for numerical values and the most frequent category for categorical data. Advanced techniques like interpolation or machine learning-based imputation can also be used for filling missing values intelligently.



4. **Explore Data Characteristics** - This step involves understanding the distribution and nature of data. Statistical analysis using measures such as mean, median, standard deviation, and skewness helps in identifying how data points are spread across different variables. Histograms, box plots, and density plots can be used to visualize data distributions and detect any imbalances. This analysis provides a foundational understanding of how different variables behave, whether they are normally distributed or contain any significant variations that need further attention.
5. **Perform Data Transformation** - Raw data often requires transformation before further analysis. This includes converting categorical data into numerical format using encoding techniques such as One-Hot Encoding or Label Encoding. Feature scaling techniques like

Min-Max Scaling or Standardization ensure that numerical variables are on a comparable scale, improving model performance. Date and time variables may also be decomposed into meaningful components such as year, month, or day of the week to extract additional insights. Data transformation ensures that the dataset is structured optimally for analysis.

6. **Visualize Data Relationships** - Data visualization is a powerful technique to identify patterns, trends, and correlations among variables. Various types of plots, such as bar charts, scatter plots, and line graphs, help in understanding relationships between features. Correlation matrices and heatmaps allow us to see how different numerical variables are related, which is useful in identifying redundant features. Through visualization, we can detect seasonality, trends, and unusual behaviors in data that may not be evident from statistical summaries alone.
7. **Handling Outliers** - Outliers are extreme values that deviate significantly from other data points and can distort analysis results. Identifying outliers can be done using box plots, histograms, or statistical methods such as the Interquartile Range (IQR) and Z-score. Once detected, outliers can be removed if they result from data entry errors, or they can be adjusted if they hold meaningful insights (e.g., high-priced luxury houses in a real estate dataset). Handling outliers correctly ensures that the analysis is not biased by extreme values.
8. **Communicate Findings & Insights** - The final step in EDA is to summarize the key observations and present them in an understandable format. This can be done through dashboards, reports, or visual storytelling using tools like Matplotlib, Seaborn, or Tableau. Effective communication of insights involves highlighting important trends, patterns, and recommendations that can guide decision-making. Whether the goal is to improve sales, optimize marketing strategies, or detect fraud, clear and concise reporting of EDA findings is crucial for making informed business and technical decisions.

Common Visualizations used in EDA:

- **Bar Charts** - Used to compare categorical data.
- **Line Graphs** – Ideal for tracking trends over time.
- **Histograms** – Show frequency distribution of numerical data.
- **Scatter Plots** – Help visualize relationships between two numerical variables.
- **Box Plots** – Useful for detecting outliers and understanding distributions.

Matplotlib:

Matplotlib is a widely used Python library for creating static, animated, and interactive visualizations. It provides extensive tools to generate a variety of plots, making it essential for Exploratory Data Analysis (EDA) and data presentation.

Key Features of Matplotlib -

- Customizable Plots – Allows modification of axes, labels, colors, and styles.
- Multiple Plot Types – Supports line plots, bar charts, scatter plots, histograms, pie charts, and more.
- Figure & Subplot Management – Enables creating multiple subplots in a single figure using `plt.subplot()`.
- Annotations & Labels – Allows adding text, legends, and annotations for better readability.
- 3D Plotting – Supports 3D visualizations using `mpl_toolkits.mplot3d`.
- Interactive Plots – Provides zooming, panning, and event handling features.
- Integration with Libraries – Works seamlessly with NumPy, Pandas, and Seaborn for data visualization.

Commonly Used Plot Types in Matplotlib -

- Line Plot (`plt.plot()`) – Shows trends over time or continuous data.
- Bar Chart (`plt.bar()`) – Compares categorical data.
- Histogram (`plt.hist()`) – Displays frequency distribution of a dataset.
- Scatter Plot (`plt.scatter()`) – Visualizes relationships between two numerical variables.
- Pie Chart (`plt.pie()`) – Represents data proportions.

EDA on Y-combinator data -**1. Score Distribution Analysis -**

The distribution of post scores is analyzed using a histogram (`plt.hist`). Helps understand how posts are rated based on their engagement and upvotes.

A histogram with 20 bins to show the spread of scores.

X-axis: Post scores, Y-axis: Number of posts.

2. Comment Activity Analysis -

The number of comments per post is analyzed using `df_comments['Link'].value_counts()`.

Identifies which posts generate the most discussion. A bar chart showing the number of comments for each post. X-axis: Post links, Y-axis: Number of comments.

3. Top 10 users by number of comments -

The dataset is grouped by the Author column to count the number of comments each user has made. The top 10 most active users are selected using `.value_counts().head(10)`. This helps identify the most engaged participants in Y Combinator discussions.

A bar chart with:

X-axis → Usernames of the top 10 commenters.

Y-axis → Number of comments made by each user.

Viridis color palette for better readability.

Rotated x-ticks to make usernames legible.

Other analysis performed in the experiment are 'Number of posts over time', 'Top 10 Most Frequent Words in Comments' and 'Number of comments vs Length of Question'